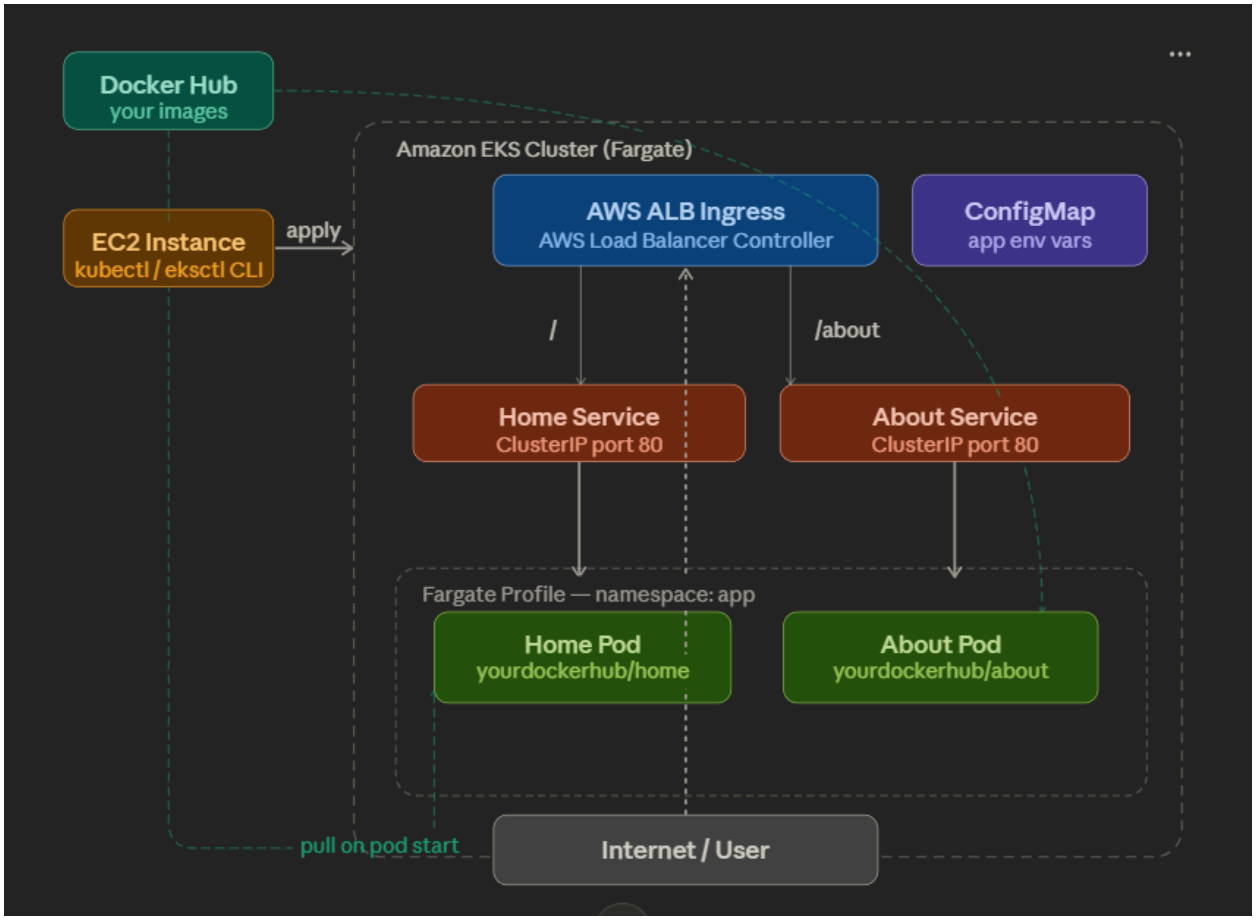


Launch an ec2 instance ubuntu instance with the ubuntu t2 small

Install all these

```
# Install kubectl
curl -LO "https://dl.k8s.io/release/$(curl -Ls
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl && sudo mv kubectl /usr/local/bin/

# Install eksctl
curl -sLO
"https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_Linux_amd64.tar.gz"
tar -xzf eksctl_Linux_amd64.tar.gz && sudo mv eksctl /usr/local/bin/

# Install AWS CLI v2 (if not already there)
```

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip && sudo ./aws/install

# Configure credentials
aws configure
```

Step 1 — Create the EKS cluster with Fargate

```
eksctl create cluster \
  --name my-cluster \
  --region us-east-1 \
  --fargate \
  --alb-ingress-access
```

```
ubuntu@ip-172-31-21-14:~$ aws s3 ls
ubuntu@ip-172-31-21-14:~$ eksctl create cluster \
  --name my-cluster \
  --region us-east-1 \
  --fargate \
  --alb-ingress-access
2026-04-08 15:38:37 [ℹ]  eksctl version 0.225.0
2026-04-08 15:38:37 [ℹ]  using region us-east-1
2026-04-08 15:38:37 [ℹ]  setting availability zones to [us-east-1b us-east-1d]
2026-04-08 15:38:37 [ℹ]  subnets for us-east-1b - public:192.168.0.0/19 private:192
2026-04-08 15:38:37 [ℹ]  subnets for us-east-1d - public:192.168.32.0/19 private:19
2026-04-08 15:38:37 [!]  Auto Mode will be enabled by default in an upcoming releas
onger be created by default. To maintain current behavior, explicitly set 'autoMode
```

This creates the cluster, VPC, subnets, and a default Fargate profile. Takes ~15 min.

bash

```
aws eks update-kubeconfig --region us-east-1 --name my-cluster
```

```
ubuntu@ip-172-31-21-14:~$ aws eks update-kubeconfig --region us-east-1 --name my-cluster
Added new context arn:aws:eks:us-east-1:379515516143:cluster/my-cluster to /home/ubuntu/.kube/config
```

Step 2 — Create a Fargate profile for your app namespace

```
eksctl create fargateprofile \
  --cluster my-cluster \
  --name app-profile \
  --namespace app \
  --region us-east-1
```

```
ubuntu@ip-172-31-21-14:~$ eksctl create fargateprofile \
  --cluster my-cluster \
  --name app-profile \
  --namespace app \
  --region us-east-1
2026-04-08 15:54:02 [ℹ]  creating Fargate profile "app-profile" on EKS cluster "my-cluster"
2026-04-08 15:54:19 [ℹ]  created Fargate profile "app-profile" on EKS cluster "my-cluster"
ubuntu@ip-172-31-21-14:~$ eksctl utils associate-iam-oidc-provider \
```

Step 3 — Install the AWS Load Balancer Controller

The ALB controller is what makes the `Ingress` resource create a real AWS ALB.

```
# Associate OIDC provider (needed for IAM roles for service accounts)
eksctl utils associate-iam-oidc-provider \
  --cluster my-cluster \
  --region us-east-1 \
  --approve
```

```
ubuntu@ip-172-31-21-14:~$ eksctl utils associate-iam-oidc-provider \
  --cluster my-cluster \
  --region us-east-1 \
  --approve
2026-04-08 15:55:11 [ℹ]  will create IAM Open ID Connect provider for cluster "my-cluster" in "us-east-1"
2026-04-08 15:55:12 [✓]  created IAM Open ID Connect provider for cluster "my-cluster" in "us-east-1"
```

```
# Download the IAM policy
curl -O
https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/main/docs/install/iam_policy.json
```

```
ubuntu@ip-172-31-21-14:~$ curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/main/docs/install/iam_policy.json
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  8955  100  8955    0     0   113k      0 --:--:-- --:--:-- --:--:--  115k
```

```
# Create the IAM policy
aws iam create-policy \
  --policy-name AWSLoadBalancerControllerIAMPolicy \
  --policy-document file://iam_policy.json
```

```
ubuntu@ip-172-31-21-14:~$ aws iam create-policy \
  --policy-name AWSLoadBalancerControllerIAMPolicy \
  --policy-document file://iam_policy.json
```

```
# Create the service account (replace 111122223333 with your AWS account ID)
eksctl create iamserviceaccount \
  --cluster my-cluster \
  --namespace kube-system \
  --name aws-load-balancer-controller \
  --attach-policy-arn arn:aws:iam::111122223333:policy/AWSLoadBalancerControllerIAMPolicy \
  --override-existing-serviceaccounts \
  --approve \
  --region us-east-1
```

```
ubuntu@ip-172-31-21-14:~$ eksctl create iamserviceaccount \
  --cluster my-cluster \
  --namespace kube-system \
  --name aws-load-balancer-controller \
  --attach-policy-arn arn:aws:iam::379515516143:policy/AWSLoadBalancerControllerIAMPolicy \
  --override-existing-serviceaccounts \
  --approve \
  --region us-east-1
2026-04-08 15:56:45 [ℹ]  1 iamserviceaccount (kube-system/aws-load-balancer-controller) was included (based on the include/exclude rules)
2026-04-08 15:56:45 [!]  metadata of serviceaccounts that exist in Kubernetes will be updated, as --override-existing-serviceaccounts was s
2026-04-08 15:56:45 [ℹ]  1 task: {
    2 sequential sub-tasks: {
        create IAM role for serviceaccount "kube-system/aws-load-balancer-controller",
        create serviceaccount "kube-system/aws-load-balancer-controller",
    } }2026-04-08 15:56:45 [ℹ]  building iamserviceaccount stack "eksctl-my-cluster-addon-iamserviceaccount-kube-system-aws-load-balancer-c
```

curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

```
ubuntu@ip-172-31-21-14:~$ curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 11929  100 11929    0     0   224k      0 --:--:-- --:--:-- --:--:--  228k
Downloading https://get.helm.sh/helm-v3.20.1-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
helm installed into /usr/local/bin/helm
ubuntu@ip-172-31-21-14:~$ helm version
version.BuildInfo{Version:"v3.20.1", GitCommit:"a2369ca71c0ef633bf6e4fccd66d634eb379b371", GitTreeState:"cle
```

helm repo update

```
ubuntu@ip-172-31-21-14:~$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "eks" chart repository
Update Complete. ⎈Happy Helming!⎈
```

Add the repo
helm repo add eks https://aws.github.io/eks-charts

```
ubuntu@ip-172-31-21-14:~$ helm repo add eks https://aws.github.io/eks-charts
"eks" has been added to your repositories
ubuntu@ip-172-31-21-14:~$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "eks" chart repository
Update Complete. ⎈Happy Helming!⎈
```

helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=my-cluster \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller \
  --set region=us-east-1 \
  --set vpcId=$(aws eks describe-cluster --name my-cluster --query "cluster.resourcesVpcConfig.vpcId" --output text)

```
ubuntu@ip-172-31-21-14:~$ helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=my-cluster \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller \
  --set region=us-east-1 \
  --set vpcId=$(aws eks describe-cluster --name my-cluster --query "cluster.resourcesVpcConfig.vpcId" --output text)
NAME: aws-load-balancer-controller
LAST DEPLOYED: Wed Apr  8 16:04:58 2026
NAMESPACE: kube-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
AWS Load Balancer controller installed!
ubuntu@ip-172-31-21-14:~$ nano app.yml
```

Now lets create a app.yml

```
# ── Namespace ──────────────────────────────────────────────────────────
apiVersion: v1
kind: Namespace
metadata:
  name: app

---
# ── ConfigMap (shared env vars for both deployments) ────────
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: app
data:
  APP_ENV: "production"
  LOG_LEVEL: "info"

---
# ── Home Deployment ────────────────────────────────────────────────────
apiVersion: apps/v1
kind: Deployment
metadata:
  name: home-deployment
  namespace: app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: home
  template:
    metadata:
      labels:
```

```yaml
      app: home
    spec:
      containers:
        - name: home
          image: allenaira/microservices:home   # ← your Docker Hub image
          ports:
            - containerPort: 80
          envFrom:
            - configMapRef:
                name: app-config

---
# ── Home Service ──────────────────────────────────────────────────────
apiVersion: v1
kind: Service
metadata:
  name: home-service
  namespace: app
spec:
  selector:
    app: home
  ports:
    - port: 80
      targetPort: 80

---
# ── About Deployment ──────────────────────────────────────────────────
apiVersion: apps/v1
kind: Deployment
metadata:
  name: about-deployment
  namespace: app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: about
  template:
    metadata:
      labels:
        app: about
    spec:
      containers:
        - name: about
```

```yaml
        image: allenaira/microservices:about  # ← your Docker Hub image
        ports:
          - containerPort: 80
        envFrom:
          - configMapRef:
              name: app-config

---
# ── About Service ──────────────────────────────────────────────────────
apiVersion: v1
kind: Service
metadata:
  name: about-service
  namespace: app
spec:
  selector:
    app: about
  ports:
    - port: 80
      targetPort: 80

---
# ── Ingress (creates an AWS ALB) ───────────────────────────────────────
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  namespace: app
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip          # required for Fargate
spec:
  rules:
    - http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: home-service
                port:
                  number: 80
          - path: /about
```

```
        pathType: Prefix
        backend:
          service:
            name: about-service
            port:
              number: 80
```

kubectl apply -f app.yml

```
ubuntu@ip-172-31-21-14:~$ kubectl apply -f app.yml
namespace/app created
configmap/app-config created
deployment.apps/home-deployment created
service/home-service created
deployment.apps/about-deployment created
service/about-service created
Warning: annotation "kubernetes.io/ingress.class" is deprecated, please use 'spec.ingressClassName' instead
ingress.networking.k8s.io/app-ingress created
```

Step 5 — Get your Load Balancer URL

kubectl get ingress app-ingress -n app

```
ubuntu@ip-172-31-21-14:~$ kubectl get ingress app-ingress -n app
NAME          CLASS    HOSTS   ADDRESS                                                                  PORTS   AGE
app-ingress   <none>   *       k8s-app-appingre-60fc5b08b1-1689917299.us-east-1.elb.amazonaws.com   80      31s
ubuntu@ip-172-31-21-14:~$
```

Check pods are Running on Fargate
kubectl get pods -n app -o wide

```
ubuntu@ip-172-31-21-14:~$ kubectl get pods -n app -o wide
NAME                                  READY   STATUS    RESTARTS   AGE   IP               NODE                                          NOMINATED NODE   READI
about-deployment-5dbf8f66df-8kfrs     1/1     Running   0          11m   192.168.125.82   fargate-ip-192-168-125-82.ec2.internal   <none>           <none
about-deployment-5dbf8f66df-bt1zh     1/1     Running   0          11m   192.168.67.154   fargate-ip-192-168-67-154.ec2.internal   <none>           <none
home-deployment-5bbcd55c75-74d5r      1/1     Running   0          11m   192.168.64.144   fargate-ip-192-168-64-144.ec2.internal   <none>           <none
home-deployment-5bbcd55c75-hmbvm      1/1     Running   0          11m   192.168.96.43    fargate-ip-192-168-96-43.ec2.internal    <none>           <none
ubuntu@ip-172-31-21-14:~$
```

Check ConfigMap values
kubectl describe configmap app-config -n app

```
home-deployment-5bbcd55c75-hmbvm    1/1     Running   0          11m   192.168.96.43    fargate-
ubuntu@ip-172-31-21-14:~$ kubectl describe configmap app-config -n app
Name:         app-config
Namespace:    app
Labels:       <none>
Annotations:  <none>

Data
====
APP_ENV:
----
production

LOG_LEVEL:
----
info


BinaryData
====

Events:  <none>
```

Check ingress rules
kubectl describe ingress app-ingress -n app

```
Events:  <none>
ubuntu@ip-172-31-21-14:~$ kubectl describe ingress app-ingress -n app
Name:             app-ingress
Labels:           <none>
Namespace:        app
Address:          k8s-app-appingre-60fc5b08b1-1689917299.us-east-1.elb.amazonaws.com
Ingress Class:    <none>
Default backend:  <default>
Rules:
  Host        Path  Backends
  ----        ----  --------
  *
              /        home-service:80 (192.168.96.43:80,192.168.64.144:80)
              /about   about-service:80 (192.168.125.82:80,192.168.67.154:80)
Annotations:  alb.ingress.kubernetes.io/scheme: internet-facing
              alb.ingress.kubernetes.io/target-type: ip
              kubernetes.io/ingress.class: alb
Events:
  Type    Reason                  Age   From     Message
  ----    ------                  ----  ----     -------
  Normal  SuccessfullyReconciled  15m   ingress  Successfully reconciled
ubuntu@ip-172-31-21-14:~$
```

Check load balancer controller logs if ALB doesn't appear
kubectl logs -n kube-system -l app.kubernetes.io/name=aws-load-balancer-controller

```
  Normal  SuccessfullyReconciled  15m   ingress  Successfully reconciled
ubuntu@ip-172-31-21-14:~$ kubectl logs -n kube-system -l app.kubernetes.io/name=aws-load-balancer-controller
{"level":"info","ts":"2026-04-08T16:05:59Z","logger":"setup","msg":"starting collect cache size"}
{"level":"info","ts":"2026-04-08T16:05:59Z","logger":"controller-runtime.metrics","msg":"Starting metrics server"}
{"level":"info","ts":"2026-04-08T16:05:59Z","logger":"controller-runtime.metrics","msg":"Serving metrics server","bindAddress":":8080
{"level":"info","ts":"2026-04-08T16:05:59Z","logger":"setup","msg":"starting collect top talkers"}
{"level":"info","ts":"2026-04-08T16:05:59Z","logger":"controller-runtime.webhook","msg":"Starting webhook server"}
{"level":"info","ts":"2026-04-08T16:05:59Z","msg":"starting server","name":"health probe","addr":"[::]:61779"}
{"level":"info","ts":"2026-04-08T16:05:59Z","logger":"controller-runtime.certwatcher","msg":"Updated current TLS certificate","cert":
erts/tls.crt","key":"/tmp/k8s-webhook-server/serving-certs/tls.key"}
{"level":"info","ts":"2026-04-08T16:05:59Z","logger":"controller-runtime.webhook","msg":"Serving webhook server","host":"","port":944
{"level":"info","ts":"2026-04-08T16:05:59Z","logger":"controller-runtime.certwatcher","msg":"Starting certificate poll+watcher","cert
-certs/tls.crt","key":"/tmp/k8s-webhook-server/serving-certs/tls.key","interval":10}
{"level":"info","ts":"2026-04-08T16:05:59Z","msg":"Attempting to acquire leader lease...","lock":"kube-system/aws-load-balancer-contr
{"level":"info","ts":"2026-04-08T16:14:53Z","msg":"Successful reconcile","tgb":{"name":"k8s-app-homeserv-f6f35e8899","namespace":"app
W8NKmdbHu8iUCD6JlVtvsi1Qtw/bKdwk15VULx7rSLxI6MuSAJJdfNzXLgvVb5oTO5VcwY"}
```

|

Change the conig file and add it into the docker hub again

allenaira/microservice:

kubectl get all -n app

```
See 'snap info <snapname>' for additional versions.
ubuntu@ip-172-31-21-14:~$ kubectl get all -n app
NAME                                    READY   STATUS    RESTARTS   AGE
pod/about-deployment-5dbf8f66df-8kfrs   1/1     Running   0          50m
pod/about-deployment-5dbf8f66df-btlzh   1/1     Running   0          50m
pod/home-deployment-5bbcd55c75-74d5r    1/1     Running   0          50m
pod/home-deployment-5bbcd55c75-hmbvm    1/1     Running   0          50m

NAME                    TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)   AGE
service/about-service   ClusterIP   10.100.69.121   <none>        80/TCP    50m
service/home-service    ClusterIP   10.100.172.34   <none>        80/TCP    50m

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/about-deployment    2/2     2            2           50m
deployment.apps/home-deployment     2/2     2            2           50m

NAME                                           DESIRED   CURRENT   READY   AGE
replicaset.apps/about-deployment-5dbf8f66df    2         2         2       50m
replicaset.apps/home-deployment-5bbcd55c75     2         2         2       50m
ubuntu@ip-172-31-21-14:~$
```

kubectl get all -n app

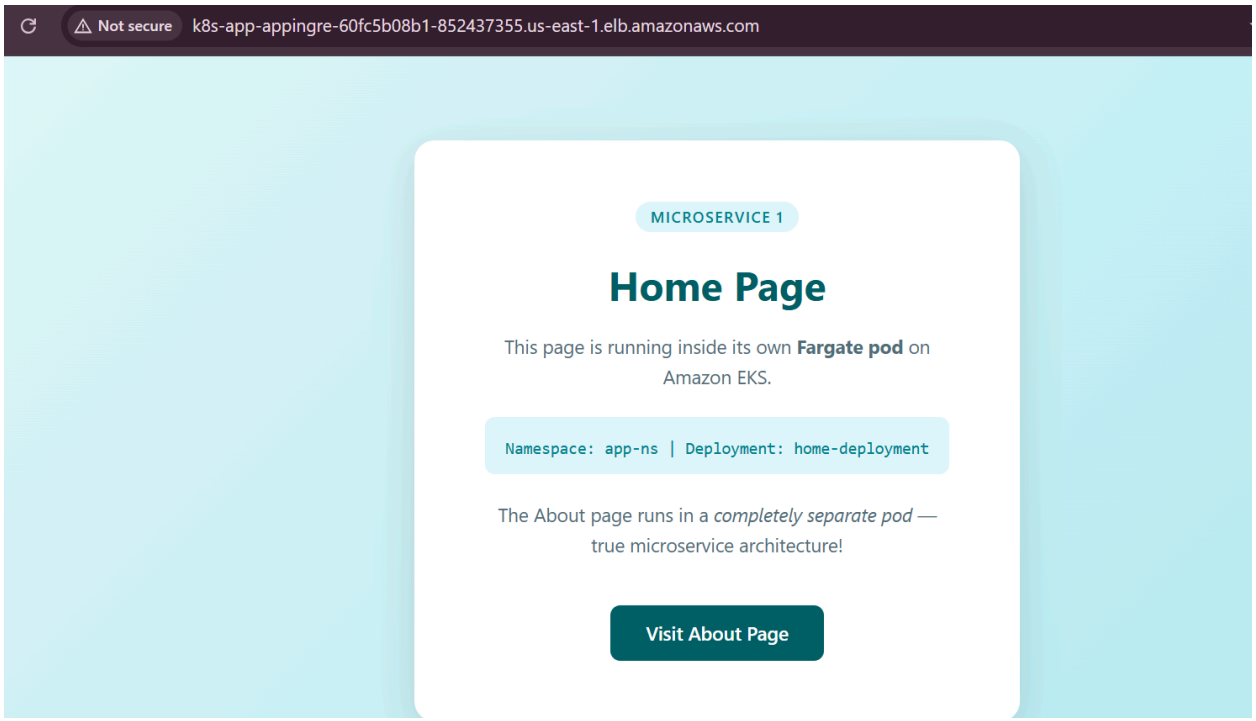
```
nginx:alpine                      645eda1c2477       93.5MB         26.9MB
ubuntu@ip-172-31-21-14:~$ kubectl get all -n app
NAME                                    READY   STATUS    RESTARTS   AGE
pod/about-deployment-5f6d6b6cfb-m4qkc   1/1     Running   0          2m1s
pod/about-deployment-5f6d6b6cfb-qdjb7   1/1     Running   0          2m57s
pod/home-deployment-5bbcd55c75-cxwn4    1/1     Running   0          10h
pod/home-deployment-5bbcd55c75-p2rqn    1/1     Running   0          10h

NAME                    TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
service/about-service   ClusterIP   10.100.2.192   <none>        80/TCP    10h
service/home-service    ClusterIP   10.100.8.236   <none>        80/TCP    10h

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/about-deployment    2/2     2            2           10h
deployment.apps/home-deployment     2/2     2            2           10h

NAME                                           DESIRED   CURRENT   READY   AGE
replicaset.apps/about-deployment-5dbf8f66df    0         0         0       10h
replicaset.apps/about-deployment-5f6d6b6cfb    2         2         2       2m57s
replicaset.apps/home-deployment-5bbcd55c75     2         2         2       10h
ubuntu@ip-172-31-21-14:~$

i-0f68fd9fbaa4d87e3 (qwerty)
```

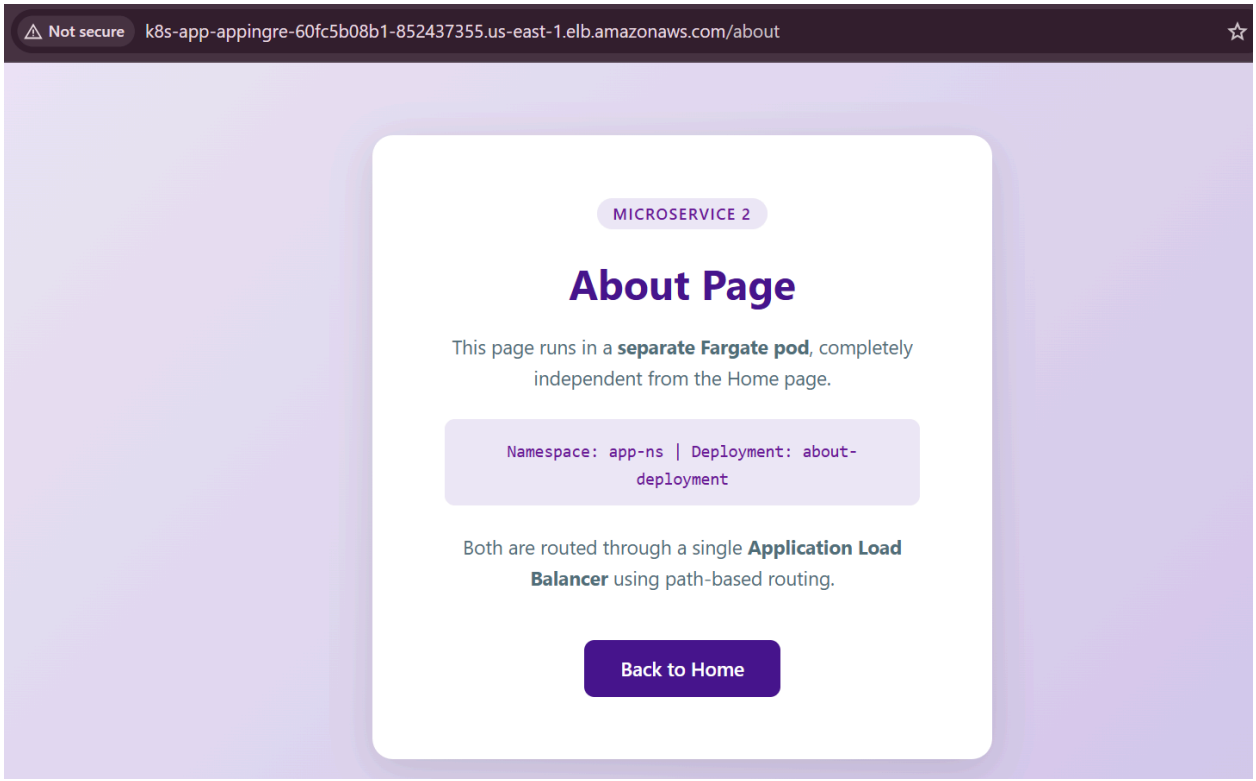
kubectl rollout restart deployment.apps/about-deployment -n app

kubectl rollout status deployment.apps/about-deployment -n app

In case of failure please run these

And you are doen





And its all done

And the concept of HPA

Now lets delete everything
kubectl delete -f app.yaml

```
ubuntu@ip-172-31-21-14:~$ kubectl delete -f app.yml
namespace "app" deleted
configmap "app-config" deleted from app namespace
deployment.apps "home-deployment" deleted from app namespace
service "home-service" deleted from app namespace
deployment.apps "about-deployment" deleted from app namespace
service "about-service" deleted from app namespace
ingress.networking.k8s.io "app-ingress" deleted from app namespace
```

Delete the IAM service account

eksctl delete iamserviceaccount \ --cluster my-cluster \ --namespace kube-system \ --name aws-load-balancer-controller \ --region us-east-1

```
ingress.networking.k8s.io "app-ingress" deleted from app namespace
ubuntu@ip-172-31-21-14:~$ eksctl delete iamserviceaccount \
  --cluster my-cluster \
  --namespace kube-system \
  --name aws-load-balancer-controller \
  --region us-east-1
2026-04-09 03:49:41 [ℹ]  1 iamserviceaccount (kube-system/aws-load-balancer-controller) was in
2026-04-09 03:49:41 [ℹ]  1 task: {
    2 sequential sub-tasks: {
        delete IAM role for serviceaccount "kube-system/aws-load-balancer-controller" [async],
        delete serviceaccount "kube-system/aws-load-balancer-controller",
    } }2026-04-09 03:49:41 [ℹ]  will delete stack "eksctl-my-cluster-addon-iamserviceaccount-k
2026-04-09 03:49:41 [ℹ]  deleted serviceaccount "kube-system/aws-load-balancer-controller"
ubuntu@ip-172-31-21-14:~$
```

Delete the Fargate profile

eksctl delete fargateprofile \ --cluster my-cluster \ --name app-profile \ --region us-east-1

```
ubuntu@ip-172-31-21-14:~$ eksctl delete fargateprofile \
  --cluster my-cluster \
  --name app-profile \
  --region us-east-1
2026-04-09 03:52:41 [ℹ]  deleted Fargate profile "app-profile" on EKS cluster "my-cluster"
ubuntu@ip-172-31-21-14:~$
```

Delete the EKS cluster (this deletes everything inside it too)

eksctl delete cluster --name my-cluster --region us-east-1

```
fargateProfileName: app-profile
ubuntu@ip-172-31-21-14:~$ eksctl delete cluster --name my-cluster --region us-east-1
2026-04-09 03:55:11 [ℹ]  deleting EKS cluster "my-cluster"
2026-04-09 03:55:11 [ℹ]  deleting Fargate profile "fp-default"
2026-04-09 03:57:19 [ℹ]  deleted Fargate profile "fp-default"
2026-04-09 03:57:19 [ℹ]  deleted 1 Fargate profile(s)
2026-04-09 03:57:19 [✓]  kubeconfig has been updated
2026-04-09 03:57:19 [ℹ]  cleaning up AWS load balancers created by Kubernetes objects of Kind Service, Ingress, or G
2026-04-09 03:57:20 [ℹ]
2 sequential tasks: { delete IAM OIDC provider, delete cluster control plane "my-cluster" [async]
}
2026-04-09 03:57:20 [ℹ]  will delete stack "eksctl-my-cluster-cluster"
2026-04-09 03:57:20 [✓]  all cluster resources were deleted
ubuntu@ip-172-31-21-14:~$ eksctl get cluster --region us-east-1
NAME            REGION          EKSCTL CREATED
my-cluster      us-east-1       True
```

It will delete the server
Wait if its not working then
aws eks delete-fargate-profile \ --cluster-name my-cluster \ --fargate-profile-name app-profile \
--region us-east-1

And you are Done!!!